Actividad de Aprendizaje

Semana 05

Indicaciones Generales:

1. El plagio se sanciona con la suspensión o expulsión del estudiante de la Universidad. Reglamento General de Estudios.
2. Sea cuidadoso con su redacción, la cual formará parte de su calificación (aspectos léxicos, sintácticos, semánticos).
3. Lea bien la pregunta o enunciado antes de proceder a su desarrollo, administre su tiempo eficazmente.

Actividad 1.

Ejercicio 1: Patrón Facade

Caso: Procesamiento de Pedidos en un Sistema de Ventas

Una tienda minorista ha desarrollado una aplicación de escritorio para registrar los pedidos realizados por sus clientes. El proceso completo de atender un pedido incluye varias tareas internas:

- Verificar si el producto solicitado tiene stock disponible.
- Calcular impuestos aplicables al pedido.
- Registrar la orden en el sistema.
- Generar un recibo de confirmación con los datos del cliente, producto, total e impuestos.

Estas operaciones están actualmente implementadas en distintas clases de servicio, pero el código del controlador que orquesta el flujo ha crecido y se ha vuelto difícil de mantener.

Se ha decidido aplicar el patrón **Facade** para encapsular este flujo completo de atención de pedido en una sola clase llamada PedidoFacade. Esta clase deberá exponer un único método público: procesarPedido(Pedido pedido), el cual coordinará internamente todos los pasos necesarios para completar el proceso de compra.

El patrón **Facade** es apropiado aquí porque:

- Permite ocultar la complejidad del subsistema de pedidos detrás de una interfaz simple.
- Facilita el mantenimiento y la evolución del sistema, manteniendo el controlador limpio.
- Permite centralizar la lógica de flujo de negocio sin exponer los detalles internos.

Requisitos Funcionales

Historia de Usuario 1: Registro de pedido

Como cliente quiero registrar un pedido para solicitar la compra de un producto.
Criterios de aceptación:

- Se debe ingresar nombre del cliente, producto y cantidad.
- La cantidad debe ser un número positivo.
- Si algún campo está vacío o inválido, se muestra un mensaje de error.

Historia de Usuario 2: Validación de stock

Como cliente **quiero recibir una alerta cuando el producto no esté disponible para** evitar generar pedidos que no pueden ser atendidos.
Criterios de aceptación:

- El sistema simula la disponibilidad en base al nombre del producto.
- Si no hay stock suficiente, se muestra el mensaje: “Stock insuficiente para el producto [nombre]”.
- El pedido no se registra si no hay stock.

Historia de Usuario 3: Cálculo del monto total

Como cliente **quiero conocer el total a pagar con impuestos para** saber el monto final antes de registrar el pedido.
Criterios de aceptación:

- El precio por unidad es fijo: S/ 20.00.
- El IGV se calcula como el 18% del subtotal.
- Se deben mostrar subtotal, IGV y total a pagar.

Historia de Usuario 4: Confirmación del registro del pedido

Como cliente **quiero confirmar el registro del pedido para** asegurar que mi solicitud ha sido procesada.
Criterios de aceptación:

- El pedido se registra solo si pasó todas las validaciones anteriores.
- Se muestra un mensaje de confirmación.

Historia de Usuario 5: Visualización del comprobante

Como cliente **quiero visualizar un comprobante del pedido para** revisar los detalles de la compra realizada.
Criterios de aceptación:

- El comprobante debe incluir: nombre del cliente, producto, cantidad, subtotal, IGV y total.
- Se muestra en pantalla después de registrar el pedido.

Ejercicio 2: Patrón Adapter

Caso: Integración con sistema de facturación electrónica externo

Luego de implementar el sistema de registro de pedidos para una tienda minorista, la empresa desea cumplir con la normativa de emisión de **comprobantes electrónicos** para cada pedido registrado. Para ello, se ha contratado a un proveedor externo que ofrece una librería (LegacyBillingSystem) para emitir boletas y facturas.

Sin embargo, dicha librería fue desarrollada años atrás y su API no sigue las convenciones ni la interfaz definida por el equipo de desarrollo actual. En lugar de un método estandarizado como emitirFactura(Cliente cliente, double monto), la clase heredada utiliza métodos con nombres incompatibles y tipos de datos no alineados con el sistema actual.

Para mantener la arquitectura limpia y compatible con el sistema de pedidos ya implementado, se decide **aplicar el patrón Adapter**, permitiendo adaptar la clase antigua (LegacyBillingSystem) a una interfaz moderna (FacturaService), sin necesidad de modificar el código legado.

Requisitos Funcionales

Historia de Usuario 1: Generación de factura electrónica

Como operador de la tienda **quiero generar una factura electrónica al registrar un pedido para** entregar al cliente un comprobante válido de su compra.

Criterios de aceptación:

- La factura se genera automáticamente al finalizar el registro del pedido.
- El pedido debe haber sido validado correctamente.
- El sistema usa una clase adaptadora que conecta con el proveedor externo.
- Se muestra un mensaje confirmando la generación exitosa de la factura.

Historia de Usuario 2: Confirmación del estado de la factura

Como operador de la tienda **quiero ver si la factura fue generada correctamente**, **para** asegurarme de que la venta se completó sin errores.

Criterios de aceptación:

- Si la generación fue exitosa, se muestra el nombre del cliente y el monto facturado.
- Si ocurre un error, se muestra un mensaje explicativo en pantalla.

Entregables:

- Prepara un documento basado en los puntos anteriores para exponerlos en clase.



Fecha límite de presentación: Determinados por el docente.
Integrantes: Determinados por el docente.